

10 Regression



10.1 From Projection to Regression

From a straight line to orthogonal projection — deriving the optimal parameters of univariate linear regression with the Gram matrix and the generalized inverse

Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Core Skills You Will Master

1

Fitting data with a straight line to express a linear relationship between variables

2

The intercept sets vertical position; the slope sets steepness

3

The predicted-value vector is the orthogonal projection of the target vector onto the span of the explanatory variables

4

Using the Gram matrix to solve for the optimal parameters

5

The role of the generalized inverse in overdetermined systems, and how it solves for regression coefficients

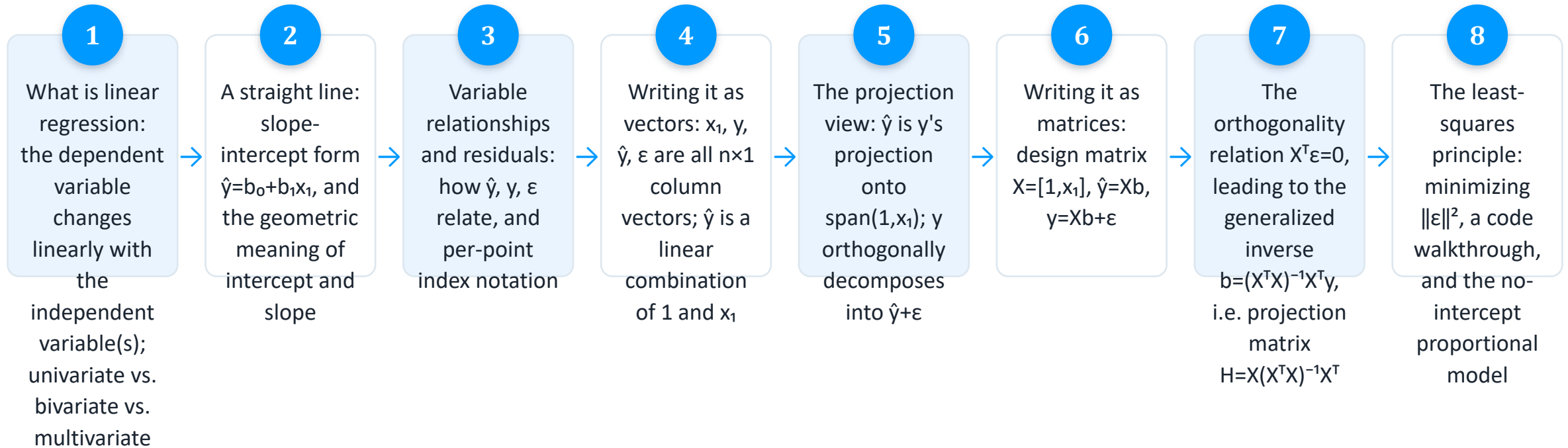
6

Both the predicted value and the residual are expressed via a projection matrix, and the two are orthogonal

7

The least-squares principle: minimizing the sum of squared residuals

From a Straight Line to Orthogonal Projection to Least Squares



Seven Lenses for Every Topic in This Chapter

- 1 Geometric view: the predicted value is the true-value vector's orthogonal projection onto the span of the explanatory variables
- 2 Algebraic view: design matrix $\mathbf{X}$, Gram matrix $\mathbf{X}^T\mathbf{X}$, generalized inverse $(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$
- 3 Optimization view: least squares — minimizing the sum of squared residuals $\|\boldsymbol{\varepsilon}\|^2$
- 4 Statistical view: residuals, goodness of fit, significance testing — left for Statistics Made Easy to cover in depth
- 5 The path of generalization: from univariate to bivariate/multivariate linear regression, on to nonlinear polynomial regression
- 6 Programming view: hand-written matrix formulas versus calling Scikit-Learn — the results fully agree
- 7 Applications: prediction, trend fitting, explaining variable relationships — one of the most basic tools in data analysis

WHAT IS LINEAR REGRESSION?

Regression Describes How a Dependent Variable Changes With an Independent Variable

Regression: build a mathematical model describing how a dependent variable changes with one or more independent variables

Linear regression's core assumption: the independent variables affect the dependent variable additively — not through exponentials, products, or other nonlinear relationships

Univariate linear regression: only one independent variable is used, e.g. predicting weight from height

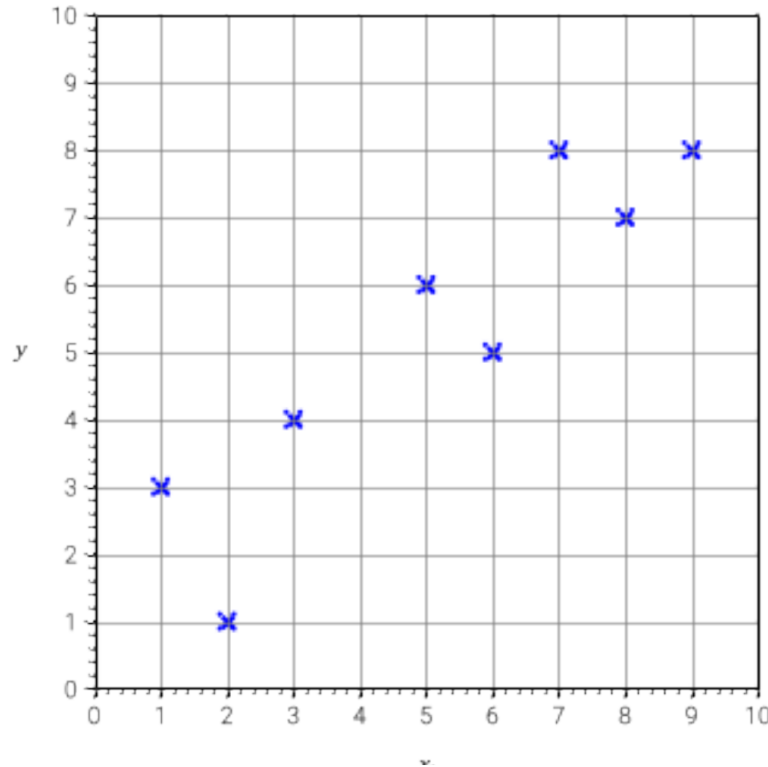
Bivariate and multivariate regression: two or more independent variables are used — the count of independent variables is what "uni-/bi-/multi-" refers to

The next section introduces polynomial regression — a common nonlinear regression model

Caution: linear regression reveals correlation only, not causation — behind it there may be a third factor, or pure coincidence

Figure 1 · Scatter Data With a Roughly Linear Trend

- 9 points trend roughly linearly; we want a line close to every point
- x_i is the independent variable; y the dependent variable



The Red Line, Written in Slope-Intercept Form

$$\hat{y} = b_0 + b_1 x_1$$

- b_0 is the intercept, b_1 is the slope — this is the slope-intercept form of a line in a plane
- b_0 is written before b_1 to match the later order of the parameter vector $\mathbf{b}=[b_0, b_1]^T$
- $\hat{y}$ reads "y-hat": y is the true value on the scatter plot, $\hat{y}$ is the line's estimated, predicted value

Figure 2 · A Red Line Fit to the Scatter Data

- The red line $\hat{y}=b_1x_1+b_0$ stays as close as possible to the blue points

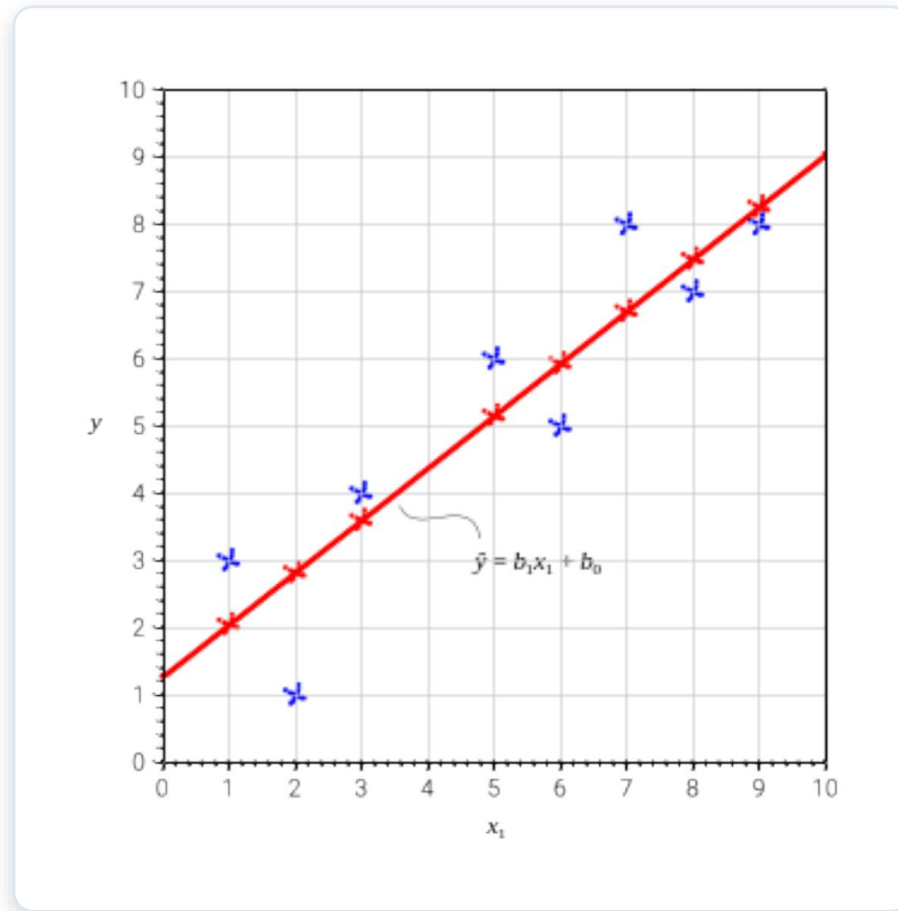
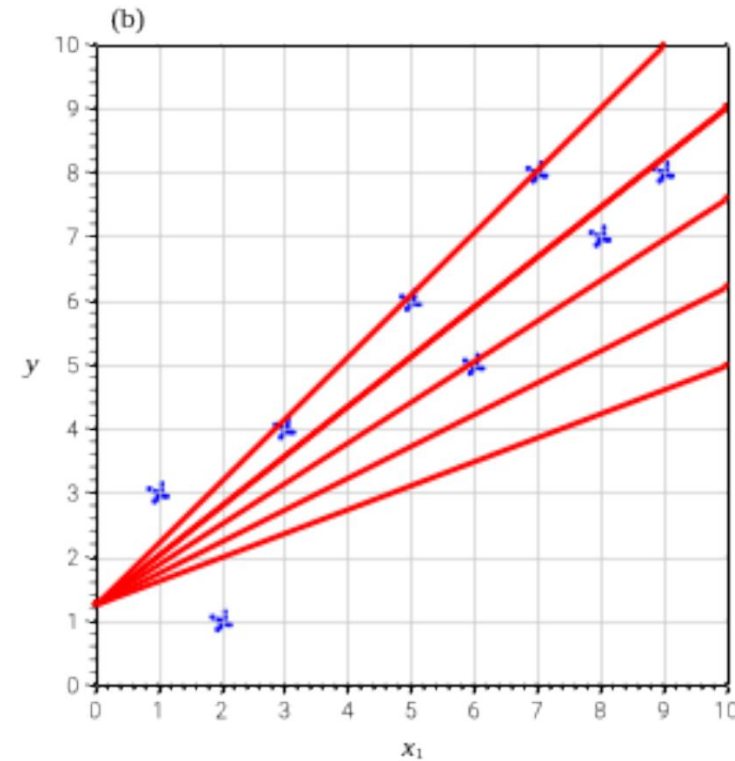
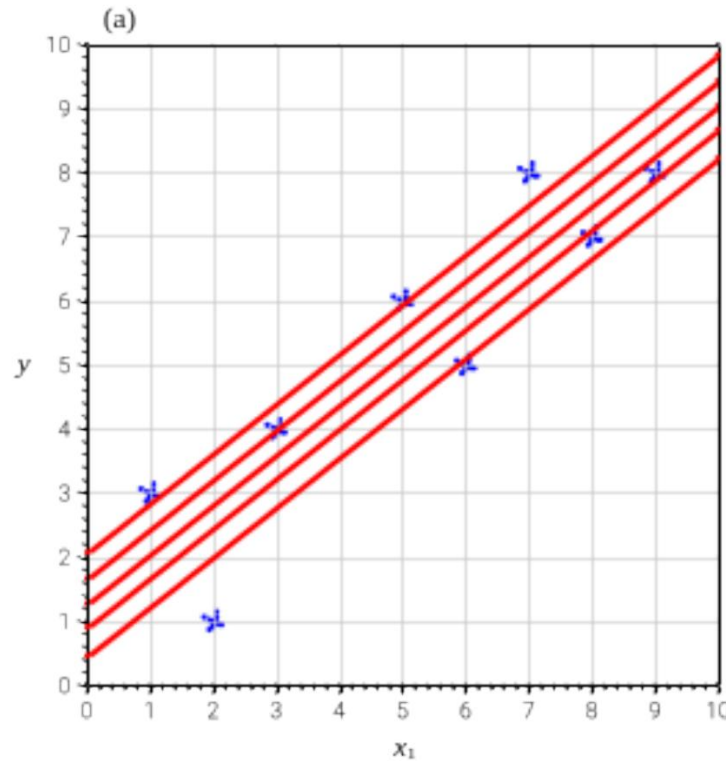


Figure 3 · (a) Intercept Sets Position, (b) Slope Sets Steepness

- (a) Slope fixed: intercept b_0 shifts the line up or down
- (b) Intercept fixed: slope b_1 pivots the line — larger $|b_1|$ is steeper



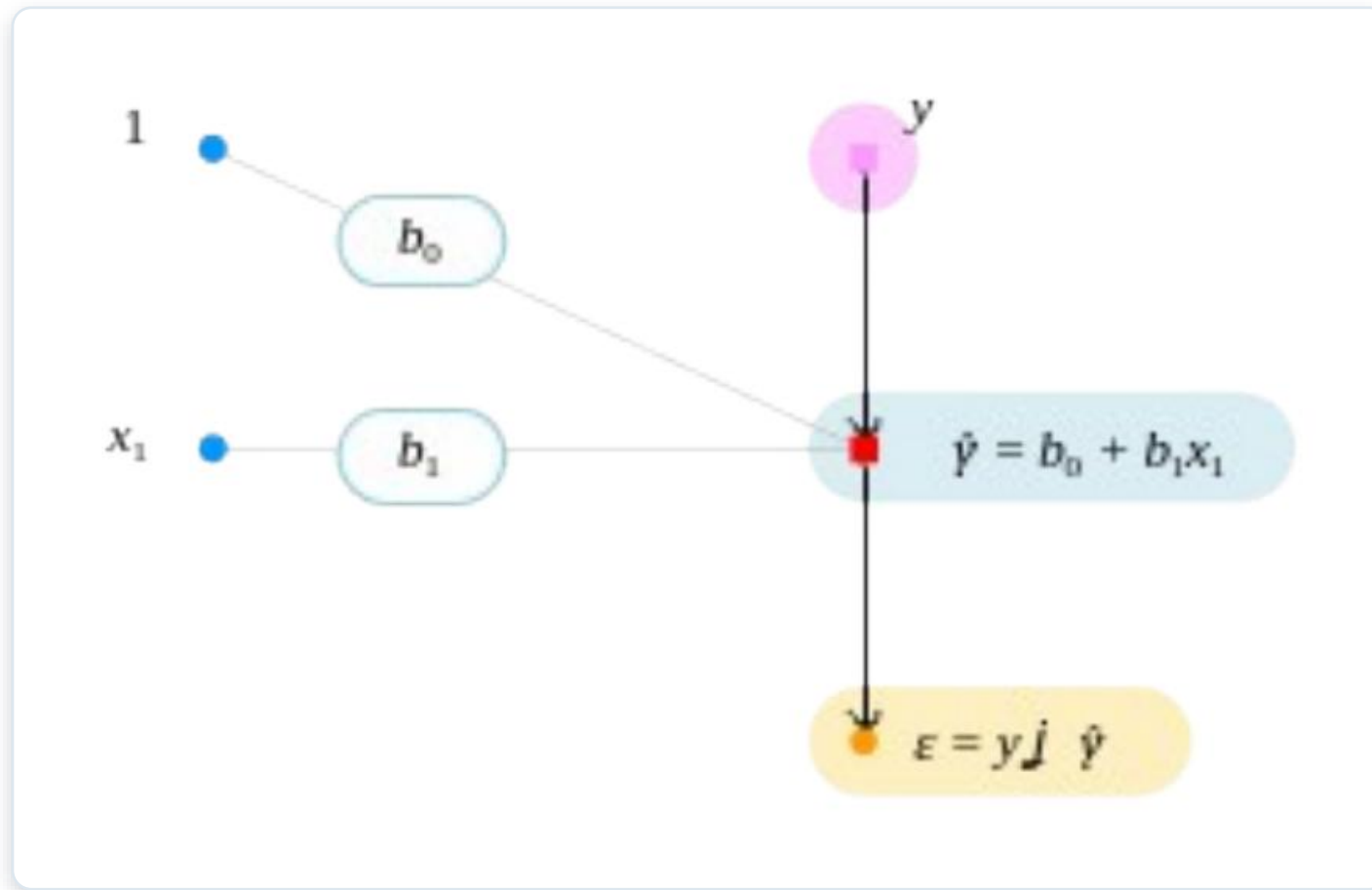
The Residual Is the True Value Minus the Predicted Value

$$\varepsilon = y - \hat{y} = y - (b_0 + b_1x_1)$$

- ε is also called the residual, error term, disturbance, or noise term
- The gap between true y and predicted $\hat{y}$ is exactly what we want to shrink as much as possible

Figure 4 · Variable Relationships in Univariate Linear Regression

- 1 and x_1 are multiplied by b_0 and b_1 and summed to get $\hat{y}$; y minus $\hat{y}$ gives the error ε



The Predicted Value, Error, and True Value at Point i

$$y^{(i)} = b_0 + b_1 x_1^{(i)} + \varepsilon^{(i)}$$

- Prediction: $\hat{y}^{(i)} = b_0 + b_1 x_1^{(i)}$; error: $\varepsilon^{(i)} = y^{(i)} - \hat{y}^{(i)} = b_0 + b_1 x_1^{(i)} - \hat{y}^{(i)}$
- True value $y^{(i)}$ and estimate $\hat{y}^{(i)}$ generally don't coincide — the vertical gap between them is the residual

Figure 5 · True Value, Estimate, and Error in Univariate Regression

- Highlighted point i : horizontal $x_1^{(i)}$, true value $y^{(i)}$; $\hat{y}^{(i)}$ is its match on the red line

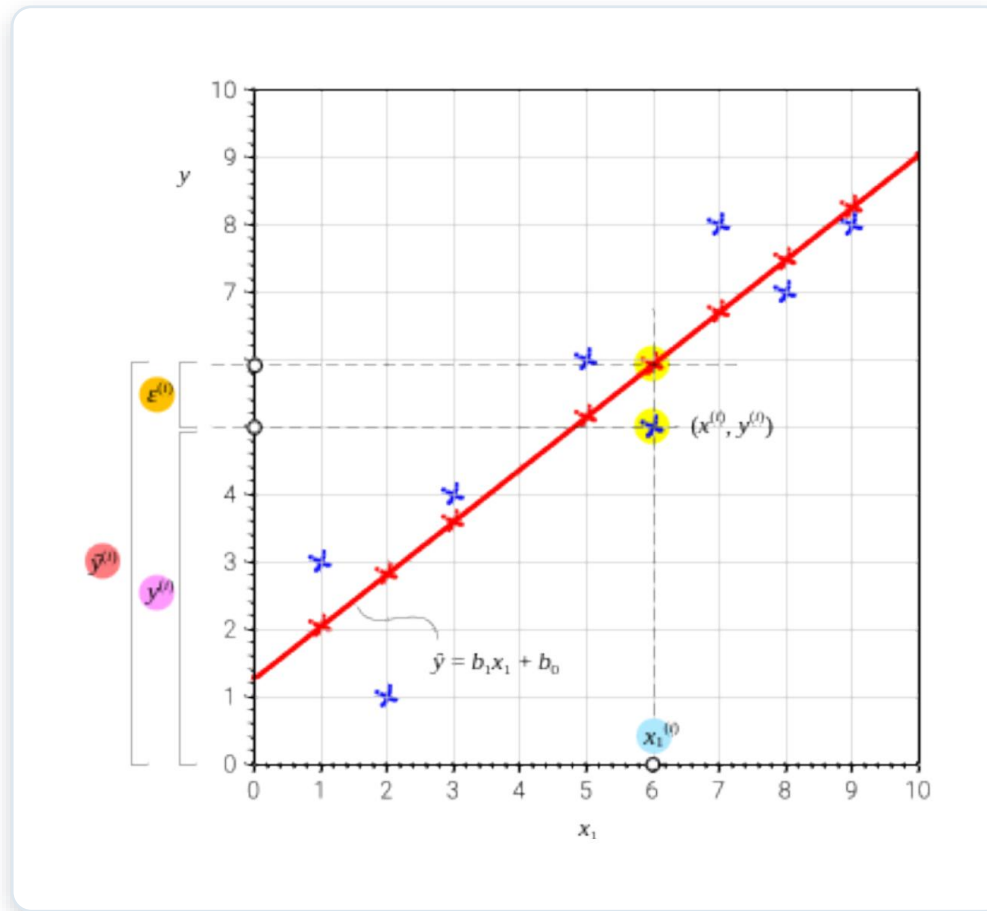
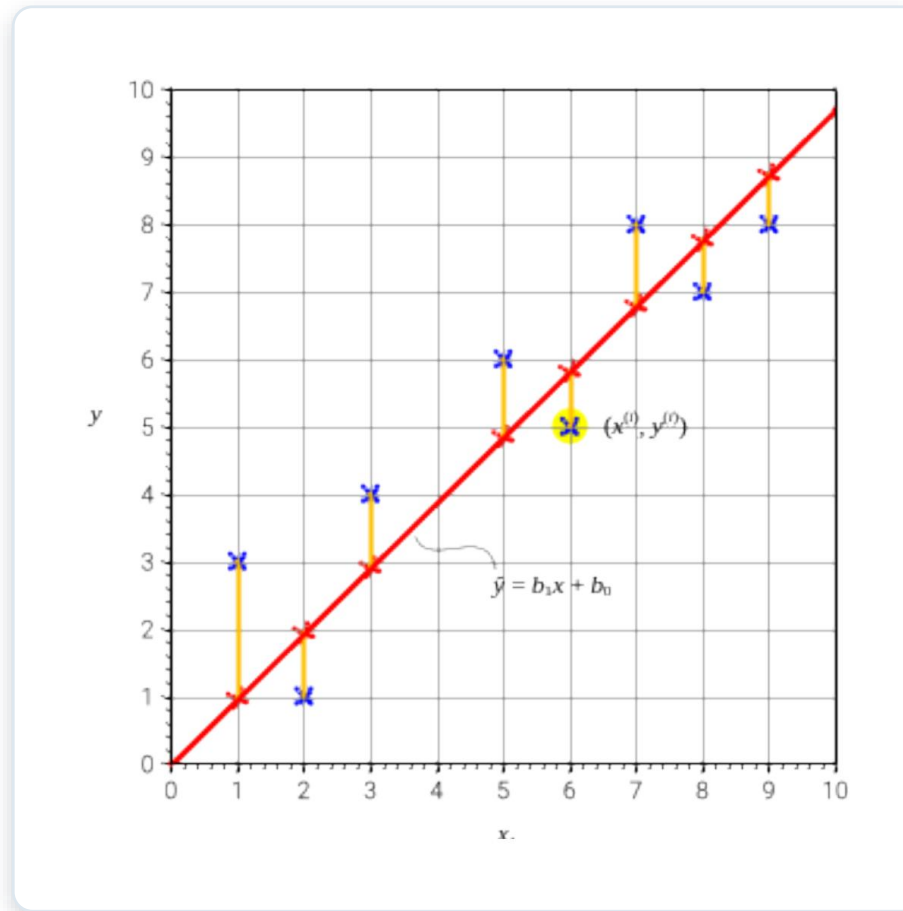


Figure 6 · The Error at Every Point (Orange Segments)

- Each orange segment is one point's residual; shrinking them all fits the red line well



The Coordinates, Estimates, and Errors All Become $n \times 1$ Column Vectors

$\mathbf{x}_1, \mathbf{y}, \hat{\mathbf{y}}, \boldsymbol{\varepsilon}$ are all $n \times 1$ column vectors

- $\mathbf{x}_1$ is the horizontal coordinate in Figure 6; $\mathbf{y}$ the blue- $\times$ vertical coordinate; $\hat{\mathbf{y}}$ the red- $\times$ vertical coordinate
- $\boldsymbol{\varepsilon}$ matches the orange segments in Figure 6 — the i -th entry of each is $x_1^{(i)}, y^{(i)}, \hat{y}^{(i)}, \varepsilon^{(i)}$

The Independent- and Dependent-Variable Column Vectors From Figure 1

$$\mathbf{x}_1 = [1, 2, 3, 5, 6, 7, 8, 9]^T, \mathbf{y} = [3, 1, 4, 6, 5, 8, 7, 8]^T$$

- Two length-9 column vectors, each entry matching one of Figure 1's 9 scattered points

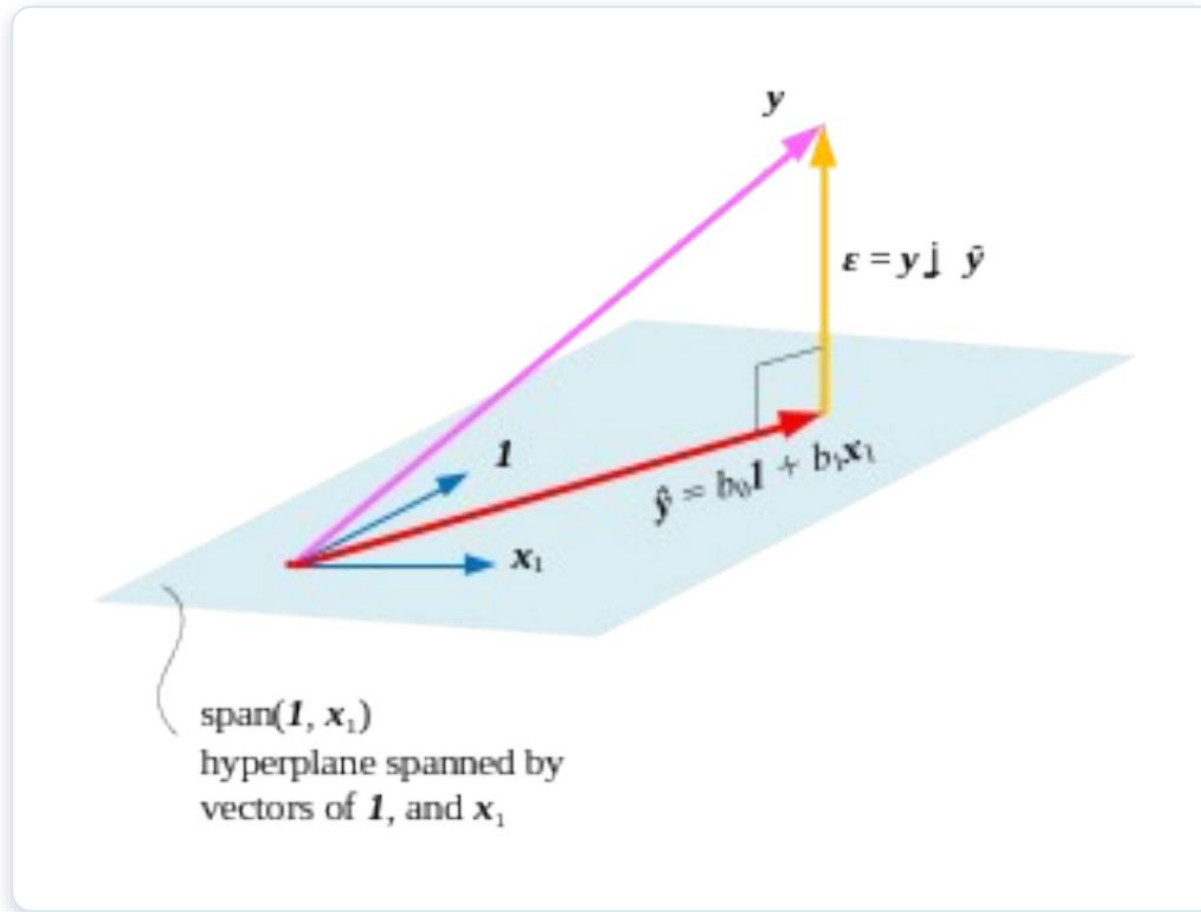
$\hat{y}$ Is a Linear Combination of $\mathbf{1}$ and $\mathbf{x}_1$

$$\hat{y} = b_0 \mathbf{1} + b_1 \mathbf{x}_1$$

- $\mathbf{1}$ is an all-ones column vector shaped like $\mathbf{x}_1$ — this is the linear combination we've seen throughout the book
- In other words, $\hat{y}$ lies in the plane $\text{span}(\mathbf{1}, \mathbf{x}_1)$ spanned by $\mathbf{1}$ and $\mathbf{x}_1$ — here $\mathbf{1}$ and $\mathbf{x}_1$ are assumed linearly independent

Figure 7 · Projecting y Onto the Plane $\text{span}(1, x_1)$

- True value y lies outside $\text{span}(1, x_1)$; its projection onto the plane is $\hat{y}$, normal component ϵ



The True Value y Decomposes Orthogonally Into Prediction $\hat{y}$ and Error ε

$$y = \hat{y} + \varepsilon = b_0 \mathbf{1} + b_1 \mathbf{x}_1 + \varepsilon$$

- Rearranging gives the error vector: $\varepsilon = y - \hat{y} = y - (b_0 \cdot \mathbf{1} + b_1 \mathbf{x}_1)$
- The error vector ε is perpendicular to the plane $\text{span}(\mathbf{1}, \mathbf{x}_1)$ — this is exactly what we need next to solve for the optimal parameters

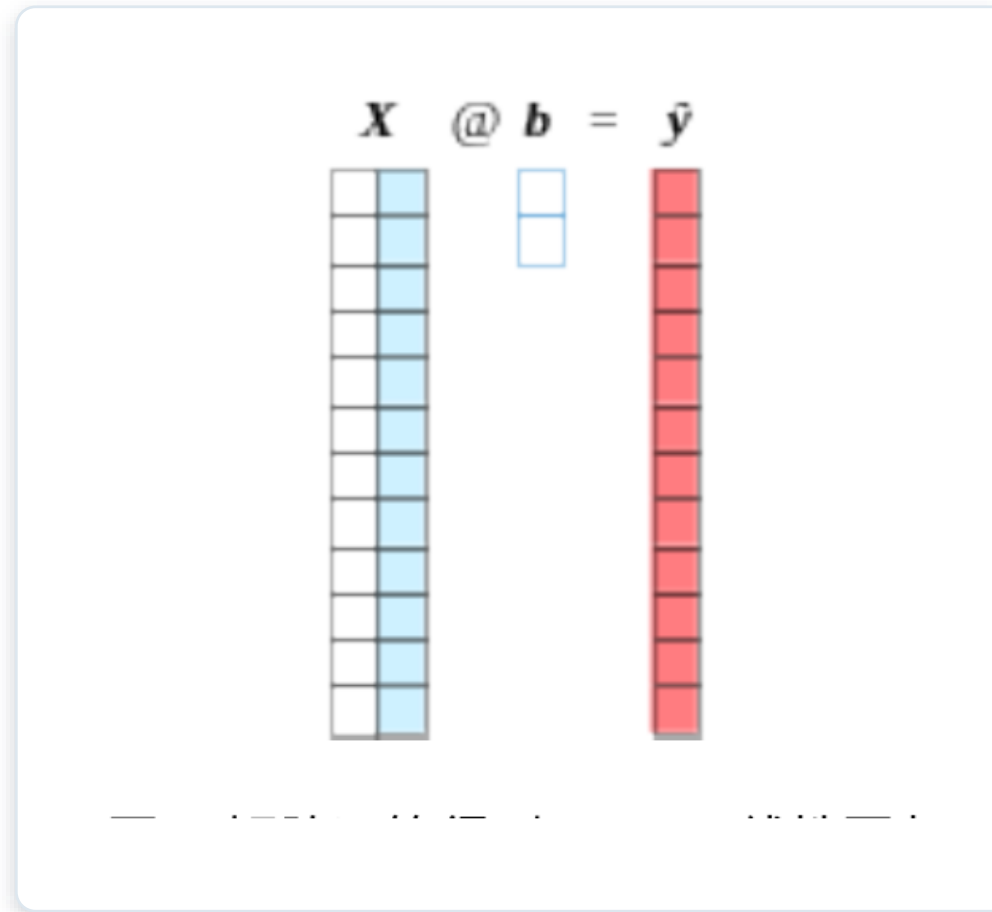
The Linear Combination Becomes Matrix Multiplication: $\hat{y} = Xb$

$$\hat{y} = [1 \ x_1] [b_0; b_1] = Xb$$

- $X=[1, x_1]$ stacks the two column vectors side by side into an $n \times 2$ matrix; $b=[b_0, b_1]^T$ is the parameter vector

Figure 8 · Matrix Multiplication Producing $\hat{y}$, Univariate Regression

- Each row of X is one sample; multiplying by column vector b gives the predicted value $\hat{y}$



X Is Called the Design Matrix

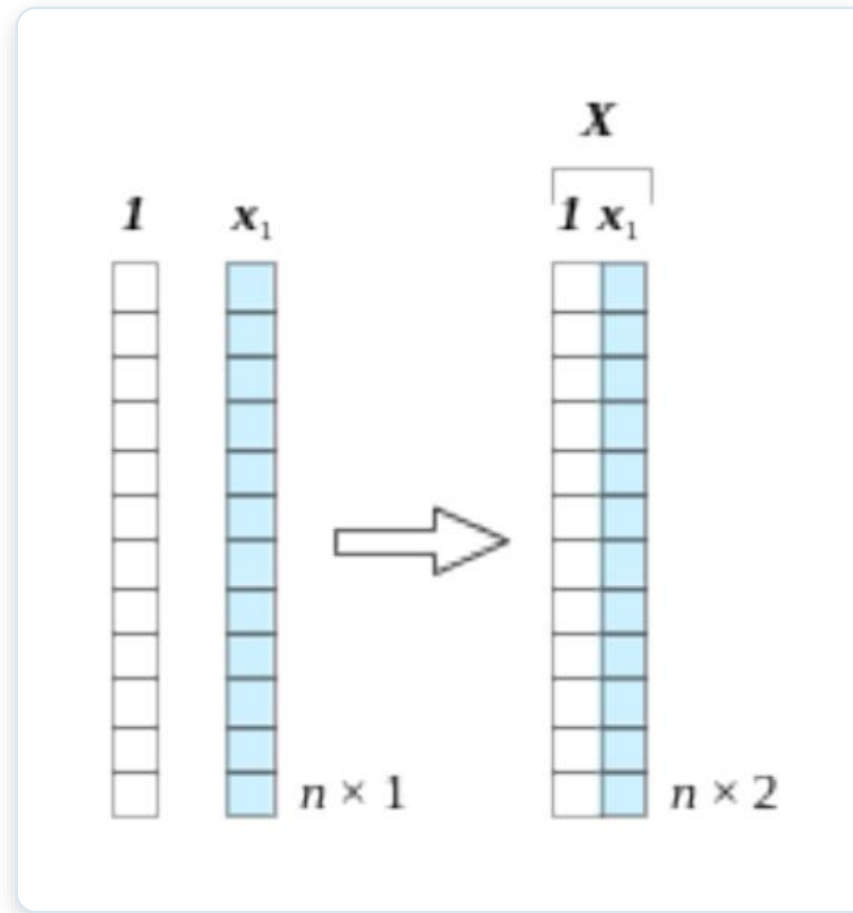
A design matrix typically adds an all-ones first column for the intercept term; the remaining columns hold the independent variables' values

Figure 1's design matrix X is an 8-row, 2-column matrix: the first column is all 1s, the second holds x_1 's 9 values

For univariate regression with a nonzero intercept, an all-ones column 1 must be added to the left of X

Figure 9 · Constructing the Design Matrix X , Univariate Regression

- Stack all-ones column $\mathbf{1}$ ($n \times 1$) with independent-variable column x_1 ($n \times 1$) to form X ($n \times 2$)



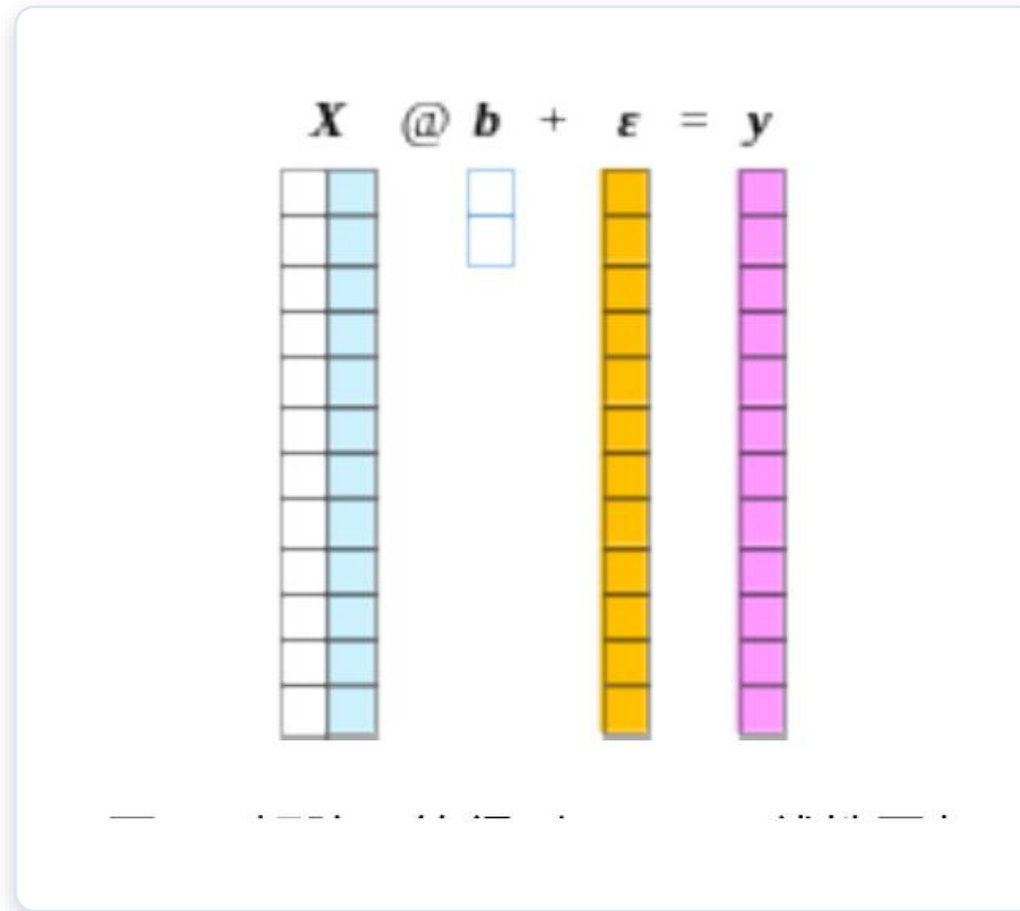
The True Value y as the Sum of Prediction $\hat{y}$ and Error ε

$$y = \hat{y} + \varepsilon = Xb + \varepsilon$$

- y is the blue- $\times$ vertical coordinate in Figure 6; $\hat{y}$ the red- $\times$ vertical coordinate; ε the orange segments

Figure 10 · Matrix Multiplication Producing y, Univariate Regression

- Adding the error column vector ϵ to Xb restores the true value y , row by row



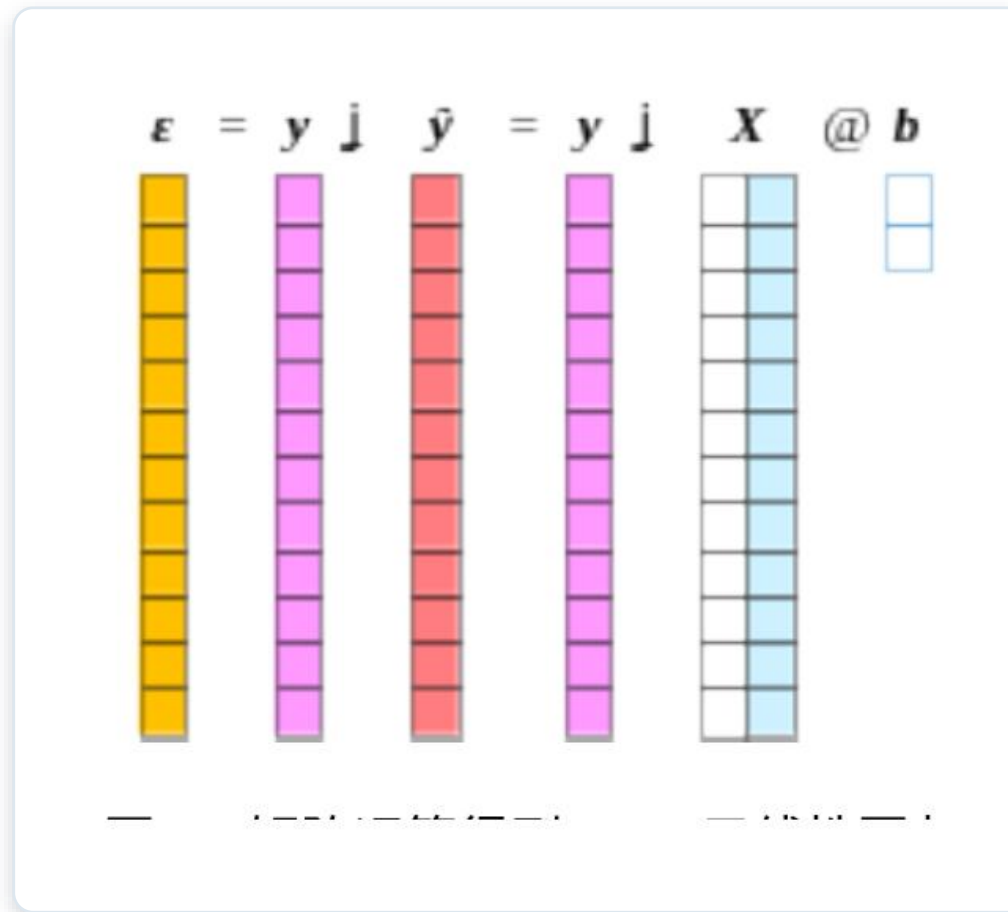
The Error Vector ε Is Perpendicular to Every Column of X

$$\varepsilon = y - \hat{y} = y - Xb$$

- Restating the earlier projection result in matrix language: ε is orthogonal to every column of design matrix X

Figure 11 · Matrix Multiplication Producing ϵ , Univariate Regression

- Subtracting Xb from y gives the residual column vector ϵ , row by row



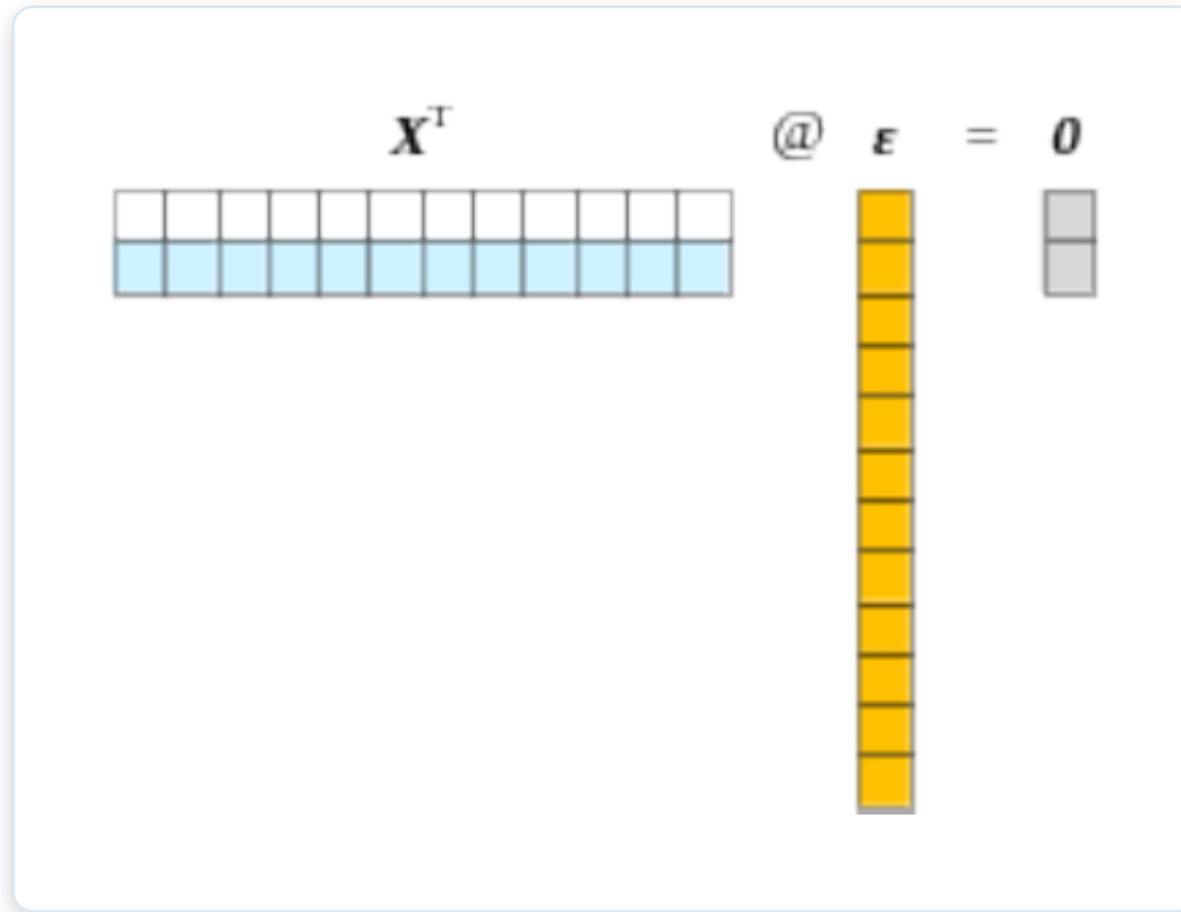
ε Perpendicular to Every Column of X , as a Matrix Product

$$\mathbf{1}^T \varepsilon = 0, \mathbf{x}_1^T \varepsilon = 0 \implies \mathbf{X}^T \varepsilon = \mathbf{0}$$

- Combining the two orthogonality conditions into one matrix product: $\mathbf{X}^T \varepsilon = \mathbf{0}$ — already covered in the previous chapter's §9.4–9.5

Figure 12 · ϵ Perpendicular to Every Column of X , Univariate Regression

- X^T times ϵ gives the zero vector — ϵ is orthogonal to every column of X



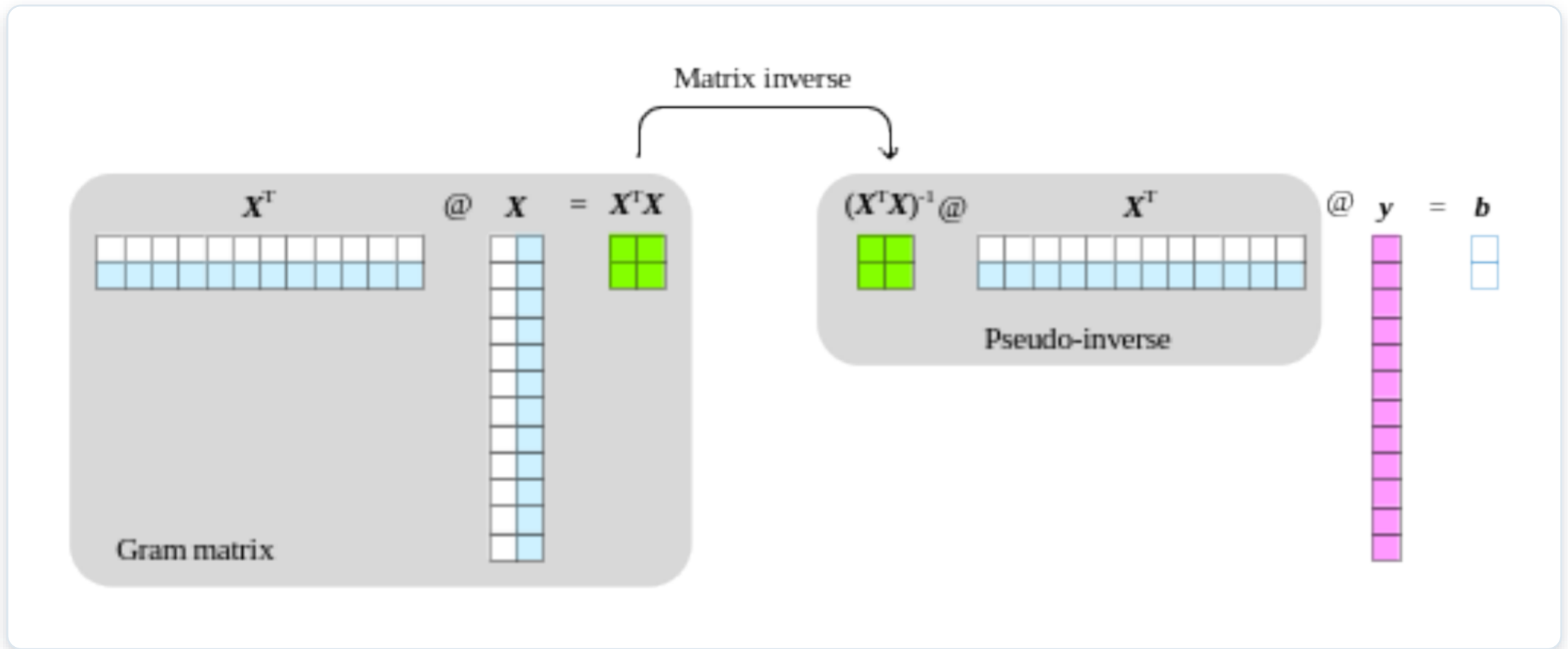
Substituting $\varepsilon = y - Xb$ Into the Orthogonality Relation Solves for b

$$X^T X b = X^T y \Rightarrow b = (X^T X)^{-1} X^T y$$

- 1 and x_1 linearly independent $\Rightarrow X$ has full column rank $\Rightarrow$ Gram matrix $X^T X$ is full rank and invertible, giving b 's closed form
- b is exactly the coordinate of y 's projection onto $\text{span}(1, x_1)$ — a scalar projection!
- $(X^T X)^{-1} X^T$ is called the pseudo-inverse, also the generalized inverse

Figure 13 · The Workflow for Computing Parameter Vector b

- Invert the Gram matrix $X^T X$, multiply by X^T for the pseudo-inverse, then multiply by y for b



Computing the Red Line's Actual Parameters From Figure 2

$$\mathbf{b} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = [8,41; 41,269]^{-1} [42; 261] = [1.267; 0.777]$$

- That is, $b_0=1.267$ and $b_1=0.777$ (results in this section are rounded to three decimal places)
- The fitted line's equation is: $\hat{y}=b_0+b_1x_1=1.267+0.777x_1$

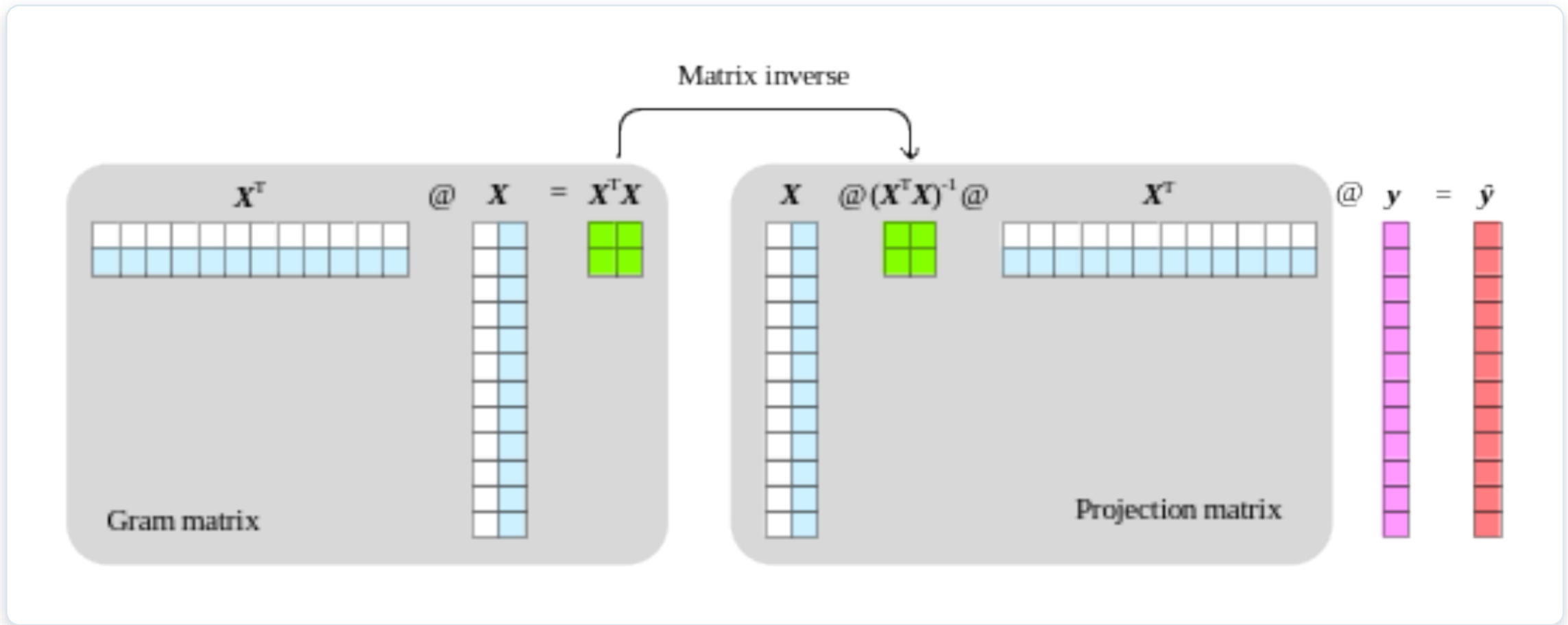
Substituting b's Closed Form Gives Projection Matrix H

$$\hat{\mathbf{y}} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} = \mathbf{H}\mathbf{y}$$

- $\mathbf{H}=\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$ is called the projection matrix — it maps $\mathbf{y}$ directly onto $\hat{\mathbf{y}}$
- Plugging in Figure 1's numbers: $\hat{\mathbf{y}}=[2.044,2.821,3.598,5.152,5.929,6.707,7.484,8.261]^T$

Figure 14 · The Matrix Multiplication That Computes $\hat{y}$, Univariate Regression

- Invert the Gram matrix, sandwich it between X and X^T for H , then multiply by y for $\hat{y}$



How Do We Quantify a "Small" Residual?

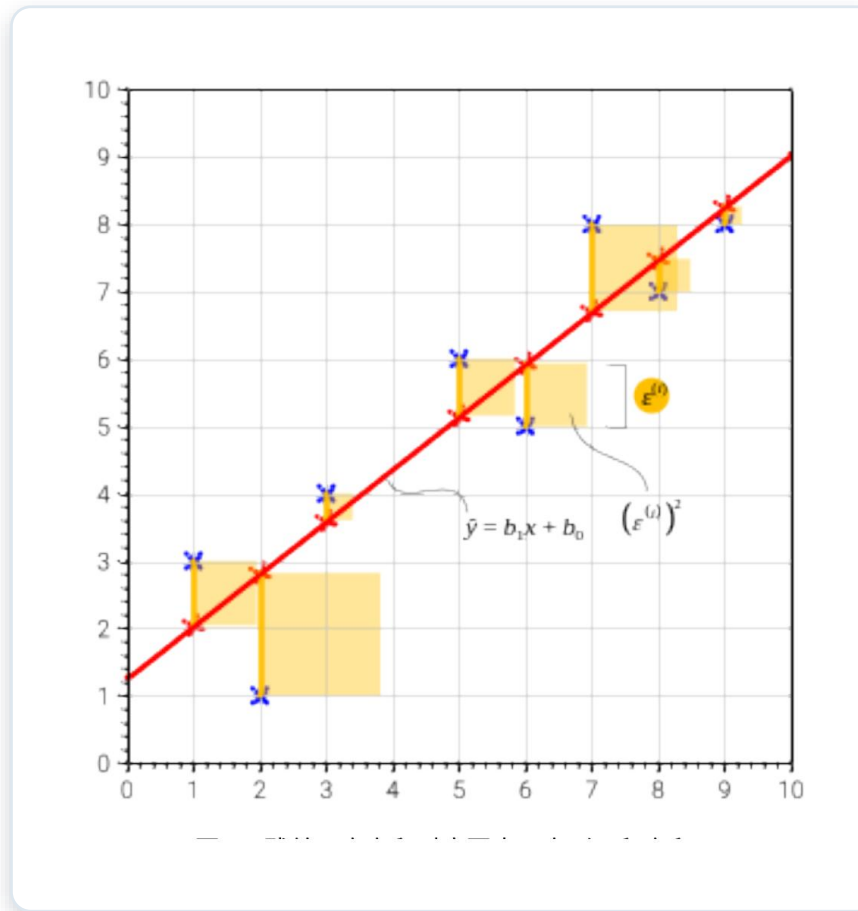
A vector norm measures a vector's "size"; the most common is the L2 norm: $\|\epsilon\| = \|y - Xb\|$

For convenience we often square the L2 norm: $\|\epsilon\|^2 = \epsilon^T \epsilon = (y - Xb)^T (y - Xb) = \sum (\epsilon^{(i)})^2$

This is ordinary least squares (OLS): minimizing the sum of squared errors between predicted and true values

Figure 15 · Sum of Squared Residuals as the Total Area of Squares

- Each square's side is a residual, so its area is $(\epsilon^{(i)})^2$; the numbers give $\|\epsilon\|^2=7.949$



How Do We Actually Minimize $\|\varepsilon\|^2$?

Solving for the minimum of $\|\varepsilon\|^2$ needs optimization tools — a topic for Calculus Made Easy

Getting a red fitted line is not the end of regression analysis — it's only the beginning

Checking model and variable significance, and whether residuals are normal and homoscedastic, is a topic for Statistics Made Easy

Solving Univariate Linear Regression in Python

Hand-write the matrix formula $b = \text{inv}(X.T@X)@X.T@y$ to solve for the optimal parameters

- (a) Create column vectors x , y : x is the independent variable, y the dependent one; `reshape(-1,1)` makes them 2-D column vectors
- (b) Build design matrix X : `np.ones_like(x)` makes an all-ones column, `np.column_stack` stacks it with x to form X
- (c) Compute parameter vector $b = \text{inv}(X^T X) @ X^T @ y$ — the standard least-squares formula
- (d) Compute predictions on x_array (dense values from 0 to 10), used to draw a smooth fitted line
- (e) Predict on the training data x , giving y_pred , to compare against the true values

```
## 初始化
import numpy as np
import matplotlib.pyplot as plt

## 数据
a x = np.array([1,2,3,5,6,7,8,9]).reshape(-1,1) # 自变量 x
  y = np.array([3,1,4,6,5,8,7,8]).reshape(-1,1) # 因变量 y

# 构造设计矩阵, 增加全1列向量
b X = np.column_stack((np.ones_like(x), x))

## 计算参数
c b = np.linalg.inv(X.T @ X) @ X.T @ y

# 格拉姆矩阵
X.T @ X

# 格拉姆矩阵的逆
np.linalg.inv(X.T @ X)

## 计算预测值
x_array = np.linspace(0,10)
X_array = np.column_stack((np.ones_like(x_array), x_array))

d y_array_pred = X_array @ b
e y_pred = X @ b

## 误差向量
f error = y - y_pred

# 误差向量的L2范数的平方
g error.T @ error
```

Training the Model Automatically With Scikit-Learn

LA_10_01_02.ipynb uses Scikit-Learn's LinearRegression to train and predict automatically

The underlying math is identical to least squares, and the results match the hand-written code exactly

LA_10_01_01.ipynb also has a short plotting section for the scatter and fitted line — simple code, worth studying on your own

A Special Case of Univariate Regression — a Model Through the Origin

If the model is a proportional function — intercept b_0 is 0 — only b_1 needs estimating, and the line passes through the origin

Here the design matrix is $X=x_1$; no all-ones column needs to be added

This makes it easy to derive a closed form for parameter b_1

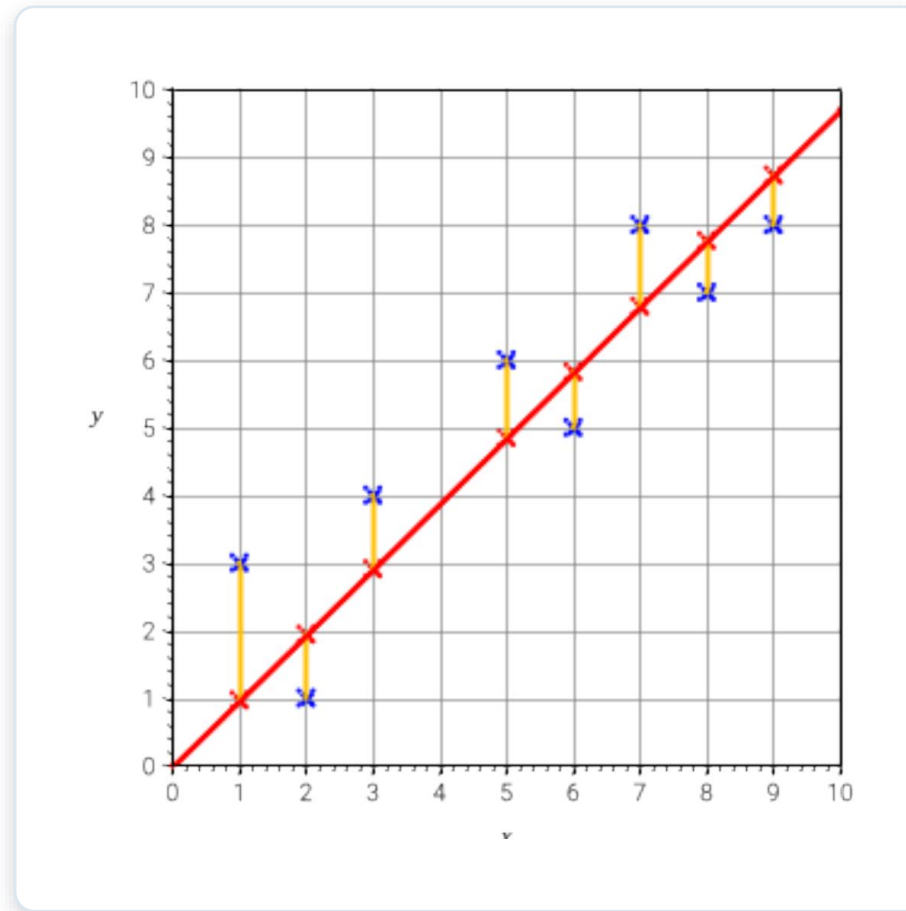
Parameter b_1 of the Proportional Function Model

$$b_1 = (\mathbf{x}_1^T \mathbf{x}_1)^{-1} \mathbf{x}_1^T \mathbf{y} = 269^{-1} \times 261 = 0.970$$

- The through-origin regression model's equation is: $\hat{y} = b_1 x_1 = 0.970 x_1$

Figure 16 · The Proportional Model, a Special Case of Univariate Regression

- The red line passes through the origin; $\hat{y}=0.970 \times [1,2,3,5,6,7,8,9]^T$, with squared-error sum 10.762



Implementing the Through-Origin Model in Code

LA_10_01_03.ipynb solves the univariate model above with a zero intercept (the proportional function)

The code structure nearly matches LA_10_01_01.ipynb — the design matrix X simply skips the all-ones column

From Projection to Regression — What Did We See?

Univariate regression $\hat{y}=b_0+b_1x_1$ is essentially a linear combination of 1 and x_1 — $\hat{y}$ lives in $\text{span}(1, x_1)$

y decomposes orthogonally into $\hat{y}+\epsilon$: $\hat{y}$ is y 's orthogonal projection onto $\text{span}(1, x_1)$, ϵ perpendicular to that plane

In matrix form $\hat{y}=Xb$, with design matrix $X=[1, x_1]$; the orthogonality relation becomes $X^T\epsilon=0$

Solving gives $b=(X^TX)^{-1}X^Ty$, where $(X^TX)^{-1}X^T$ is the generalized inverse; $H=X(X^TX)^{-1}X^T$ is the projection matrix

The least-squares principle: minimize the sum of squared residuals $\|\epsilon\|^2=\epsilon^T\epsilon$ — that is, OLS

The through-origin proportional model: design matrix drops the all-ones column, $b_1=(x_1^Tx_1)^{-1}x_1^Ty$

Key Terms in This Section

Linear Regression *A linear model describing how a dependent variable changes with independent variables*

Intercept, Slope *Intercept b_0 / Slope b_1 — set the line's vertical position and steepness*

Design Matrix X — *its first column is usually all 1s, the rest hold the independent variables*

Generalized Inverse *Also pseudo-inverse — $(X^T X)^{-1} X^T$*

Ordinary Least Squares *OLS — minimizes the sum of squared residuals $\|\epsilon\|^2$ to solve for the parameters*

Independent / Dependent Variable x_1 is the independent variable, y the dependent variable

Residual ϵ — *the difference between true and predicted values, $\epsilon = y - \hat{y}$*

Gram Matrix $X^T X$ — *used in regression to solve for the optimal parameters*

Projection Matrix H — $H = X(X^T X)^{-1} X^T$, satisfying $\hat{y} = Hy$

L2 Norm *A common measure of vector length; $\|\epsilon\|^2$ is its square*

Further Reading

Review · §9.4–9.5 Gram–Schmidt Orthogonalization, QR Decomposition

Previous chapter — the algebraic tools of orthogonal projection and the Gram matrix

Further Reading · Calculus Made Easy

How optimization tools solve for the minimum of $\|\epsilon\|^2$

Further Reading · Statistics Made Easy

Significance testing, confidence intervals, goodness of fit, multicollinearity

Open-Source Repository

All code and figures for the book: <https://github.com/Visualize-ML>

The Overdetermined System Used in Q1

Design Matrix	Parameter Vector	Observation Vector
$[[1,1],[1,2],[1,3],[1,4]]$	$[b_0,b_1]^T$	$[2,1,4,3]^T$

The system $[1,1;1,2;1,3;1,4]@ [b_0,b_1]^T=[2,1,4,3]^T$ is overdetermined (more equations than unknowns); find its least-squares solution.

Q1 – Q9

Q1 Solve the overdetermined system above by hand, and write code to find the least-squares solution b_0 , b_1 .

Q3 Use matplotlib to plot the scatter data from Q2 with the fitted line overlaid, and observe the fit.

Q5 Write a Python function `fit_line(x, y)` returning b_0 and b_1 , and test it on 3 different datasets.

Q7 Compute the projection matrix $H = X(X^T X)^{-1} X^T$ for Figure 1's data, and verify $H@y$ equals Figure 1's fitted values $\hat{y}$.

Q9 Verify: the residual vector $\epsilon = y - Xb$ satisfies $X^T \epsilon = 0$; compute this on Figure 1's data and confirm the numerical error is within $1e-10$.

Q2 Complete the code: given `np.random.rand(50)*10`, `true_slope`, and `true_intercept` generating sample data, build a univariate regression model.

Q4 Prove: if 1 and x_1 are linearly dependent (x_1 is a constant vector), the Gram matrix $X^T X$ is not invertible, and the regression coefficients cannot be uniquely determined.

Q6 Verify: does the hand-written formula $b = (X^T X)^{-1} X^T y$ agree with Scikit-Learn's `LinearRegression` on the same data?

Q8 Verify: the projection matrix H satisfies $H@H=H$ (idempotence), and confirm it in code on Figure 1's data.

Q10 – Q17

Q10 Explore: if regression uses only two observations of a single independent variable ($n=2$), what happens to the fitted line and residuals?

Q12 Compare: on the same dataset, how does a model with $b_0=0$ (proportional) differ in fit from one with a free intercept?

Q14 Explore: add an obvious outlier to the data and observe how the least-squares line gets pulled toward it.

Q16 Derive: substitute $\epsilon=y-Xb$ into $\|\epsilon\|^2=\epsilon^T\epsilon$, expand it to $y^Ty-2b^TX^Ty+b^TX^Txb$, and differentiate with respect to b to verify the normal equation.

Q11 Write a Python function `residual_sum_of_squares(y, y_pred)` that computes and returns $\|\epsilon\|^2$.

Q13 Prove that the predicted-value vector $\hat{y}$ and the residual vector ϵ are orthogonal, i.e. $\hat{y}^T\epsilon=0$.

Q15 Write Python code that colors each point in Figure 6 by the sign of its residual, and observe the distribution of positive vs. negative residuals.

Q17 Write a Python function `r_squared(y, y_pred)` implementing the goodness of fit $R^2=1-SS_{\text{res}}/SS_{\text{tot}}$ (hint: you may need to self-study this first).

Q18 – Q25

Q18 Explore: fit a univariate regression with `numpy.polyfit(x, y, 1)` and compare against this section's hand-written formula.

Q20 Explore: how does adding more columns to design matrix X affect the Gram matrix $X^T X$'s shape and how hard it is to invert?

Q22 Explore: this section assumes 1 and x_1 are linearly independent — what happens if the independent variable itself is a constant column? Verify with code.

Q24 Preview thought: with two independent variables x_1, x_2 , what shape does design matrix X become? Does the normal equation still hold?

Q19 Write Python code to visualize a heatmap of the projection matrix H (an 8×8 matrix), and observe its structure.

Q21 Write a Python function `standardize(x)` that standardizes the independent variable before fitting, and compare the resulting b_0, b_1 against the original fit.

Q23 Fit three datasets generated with different slopes and intercepts, and compare whether the Gram matrix $X^T X$'s structure changes with the data.

Q25 Preview thought: can polynomial regression $\hat{y} = b_0 + b_1 x_1 + b_2 x_1^2$ be viewed as linear regression under some design matrix? Explain your reasoning.

10 Prompts to Explore With AI

- 1 Explain why the predicted value $\hat{y}$ in univariate linear regression can be understood as y 's orthogonal projection onto $\text{span}(1, x_1)$.
- 2 Explain why the design matrix X needs an all-ones first column, and what that column represents.
- 3 Explain how the normal equation $X^T X b = X^T y$ is derived from the orthogonality relation $X^T \epsilon = 0$.
- 4 Explain why $(X^T X)^{-1} X^T$ is called the pseudo-inverse or generalized inverse, and how it relates to the ordinary inverse of a square matrix.
- 5 Use Python to demonstrate whether the hand-written least-squares formula and Scikit-Learn's LinearRegression agree.

10 Prompts to Explore With AI

6 Explain why the projection matrix $H = X(X^T X)^{-1} X^T$ satisfies $H^2 = H$, and what that means geometrically.

7 Explain why least squares minimizes the sum of squared residuals rather than the sum of absolute residuals.

8 Explain how the design matrix differs between a through-origin proportional model and a general model with an intercept.

9 Explain why the residual ϵ and the prediction $\hat{y}$ are orthogonal, and what that means for evaluating a model.

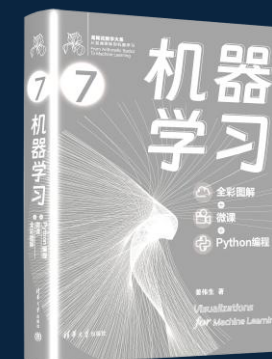
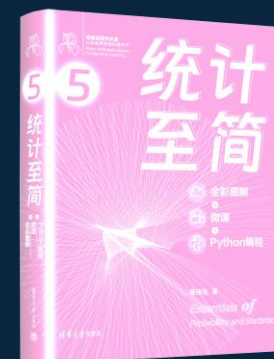
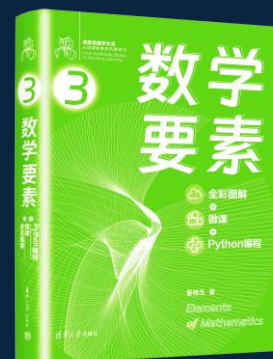
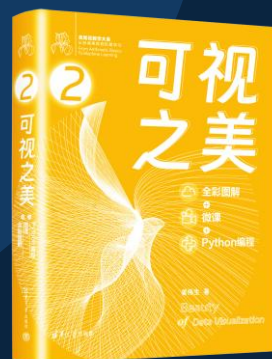
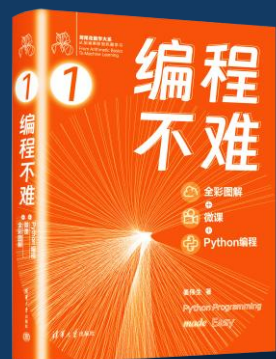
10 Explain why linear regression cannot establish causation, only correlation.

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

